

In [1]: *## Step 1: Clone Repository & Setup Environment*

```
# Clone repository from GitHub
from google.colab import userdata
import os

# Retrieve GitHub PAT from Colab secrets
github_token = userdata.get('GITHUB_TOKEN')

# Clone the repository
!git clone https://oauth2:$github_token@github.com/NTHung2034/PP-Final_Project.git

# Change to project directory
os.chdir('PP-Final_Project')
print("✓ Repository cloned successfully")
```

```
Cloning into 'PP-Final_Project'...
remote: Enumerating objects: 1591, done.
remote: Counting objects: 100% (278/278), done.
remote: Compressing objects: 100% (234/234), done.
remote: Total 1591 (delta 120), reused 159 (delta 40), pack-reused 1313 (from 2)
Receiving objects: 100% (1591/1591), 153.44 MiB | 15.15 MiB/s, done.
Resolving deltas: 100% (813/813), done.
✓ Repository cloned successfully
```

In [2]: *# Download CIFAR-10 binary dataset*
!bash ./scripts/download_cifar10.sh

```
Downloading CIFAR-10 dataset...
Data directory: ./data
--2025-12-25 13:38:35-- https://www.cs.toronto.edu/~kriz/cifar-10-binary.tar.gz
Resolving www.cs.toronto.edu (www.cs.toronto.edu)... 128.100.3.30
Connecting to www.cs.toronto.edu (www.cs.toronto.edu)|128.100.3.30|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 170052171 (162M) [application/x-gzip]
Saving to: 'cifar-10-binary.tar.gz'

cifar-10-binary.tar 100%[=====] 162.17M 70.0MB/s in 2.3s

2025-12-25 13:38:38 (70.0 MB/s) - 'cifar-10-binary.tar.gz' saved [170052171/170052171]

CIFAR-10 dataset downloaded and extracted to ./data
Files:
-rw-r--r-- 1 2156 1103 30730000 Jun 4 2009 data_batch_1.bin
-rw-r--r-- 1 2156 1103 30730000 Jun 4 2009 data_batch_2.bin
-rw-r--r-- 1 2156 1103 30730000 Jun 4 2009 data_batch_3.bin
-rw-r--r-- 1 2156 1103 30730000 Jun 4 2009 data_batch_4.bin
-rw-r--r-- 1 2156 1103 30730000 Jun 4 2009 data_batch_5.bin
-rw-r--r-- 1 2156 1103 30730000 Jun 4 2009 test_batch.bin
```

In [3]: *# Create build directory and compile CPU targets only*
!mkdir -p build
!cd build && cmake .. && make

print("\n✓ CPU implementation compiled successfully")

```

-- The CXX compiler identification is GNU 11.4.0
-- The C compiler identification is GNU 11.4.0
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: /usr/bin/c++ - skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /usr/bin/cc - skipped
-- Detecting C compile features
-- Detecting C compile features - done
-- Could not find nvcc, please set CUDAToolkit_ROOT.
CMake Warning at CMakeLists.txt:32 (message):
  CUDA requested but not found - building CPU-only version

```

```

-- Configuring done (1.8s)
-- Generating done (0.0s)
-- Build files have been written to: /content/PP-Final_Project/build
[ 3%] Building CXX object CMakeFiles/train_cpu.dir/src/train_cpu.cpp.o
[ 6%] Building CXX object CMakeFiles/train_cpu.dir/src/data/cifar10_loader.cpp.o
[ 9%] Building CXX object CMakeFiles/train_cpu.dir/src/layers/conv2d_cpu.cpp.o
[ 12%] Building CXX object CMakeFiles/train_cpu.dir/src/layers/relu_cpu.cpp.o
[ 16%] Building CXX object CMakeFiles/train_cpu.dir/src/layers/maxpool_cpu.cpp.o
[ 19%] Building CXX object CMakeFiles/train_cpu.dir/src/layers/upsample_cpu.cpp.o
[ 22%] Building CXX object CMakeFiles/train_cpu.dir/src/models/autoencoder_cpu.cpp.o
[ 25%] Building CXX object CMakeFiles/train_cpu.dir/src/utils/logger.cpp.o
[ 29%] Building CXX object CMakeFiles/train_cpu.dir/src/utils/image_utils.cpp.o
[ 32%] Building CXX object CMakeFiles/train_cpu.dir/src/utils/memory_tracker.cpp.o
[ 35%] Linking CXX executable train_cpu
[ 35%] Built target train_cpu
[ 38%] Building CXX object CMakeFiles/extract_cpu.dir/src/models_svm/extract_cpu.cpp.o
[ 41%] Building CXX object CMakeFiles/extract_cpu.dir/src/data/cifar10_loader.cpp.o
[ 45%] Building CXX object CMakeFiles/extract_cpu.dir/src/layers/conv2d_cpu.cpp.o
[ 48%] Building CXX object CMakeFiles/extract_cpu.dir/src/layers/relu_cpu.cpp.o
[ 51%] Building CXX object CMakeFiles/extract_cpu.dir/src/layers/maxpool_cpu.cpp.o
[ 54%] Building CXX object CMakeFiles/extract_cpu.dir/src/layers/upsample_cpu.cpp.o
[ 58%] Building CXX object CMakeFiles/extract_cpu.dir/src/models/autoencoder_cpu.cpp.o
[ 61%] Linking CXX executable extract_cpu
[ 61%] Built target extract_cpu
[ 64%] Building CXX object CMakeFiles/train_svm_cpu.dir/src/models_svm/train_svm_cpu.cpp.o
[ 67%] Building CXX object CMakeFiles/train_svm_cpu.dir/src/data/cifar10_loader.cpp.o
[ 70%] Building CXX object CMakeFiles/train_svm_cpu.dir/external/svm.cpp.o
[ 74%] Linking CXX executable train_svm_cpu
[ 74%] Built target train_svm_cpu
[ 77%] Building CXX object CMakeFiles/extract_features_cpu.dir/src/models_svm/extract_cpu.cpp.o
[ 80%] Building CXX object CMakeFiles/extract_features_cpu.dir/src/data/cifar10_loader.cpp.o
[ 83%] Building CXX object CMakeFiles/extract_features_cpu.dir/src/layers/conv2d_cpu.cpp.o
[ 87%] Building CXX object CMakeFiles/extract_features_cpu.dir/src/layers/relu_cpu.cpp.o
[ 90%] Building CXX object CMakeFiles/extract_features_cpu.dir/src/layers/maxpool_cpu.cpp.o
[ 93%] Building CXX object CMakeFiles/extract_features_cpu.dir/src/layers/upsample_cpu.cpp.o
[ 96%] Building CXX object CMakeFiles/extract_features_cpu.dir/src/models/autoencoder_cpu.cpp.o
[100%] Linking CXX executable extract_features_cpu
[100%] Built target extract_features_cpu

✓ CPU implementation compiled successfully

```

In [4]: !./build/train_cpu

```
=== CIFAR-10 Autoencoder (CPU) ===  
Config: 500 images, 2 epochs, batch size=64, learning rate=0.001
```

```
Loading CIFAR-10 dataset...  
Loading CIFAR-10 training data...  
  Loaded data_batch_1.bin  
  Loaded data_batch_2.bin  
  Loaded data_batch_3.bin  
  Loaded data_batch_4.bin  
  Loaded data_batch_5.bin  
Training data loaded: 50000 images  
Data loading completed in 0.90s  
Memory after data load: 619.09 MB  
Memory after model init: 619.22 MB
```

```
--- Epoch 1/2 ---
```

```
Batch 1/8 - Loss: 1.092259  
Batch 2/8 - Loss: 0.491947  
Batch 3/8 - Loss: 0.285188  
Batch 4/8 - Loss: 0.223967  
Batch 5/8 - Loss: 0.211190  
Batch 6/8 - Loss: 0.209135  
Batch 7/8 - Loss: 0.190010  
Batch 8/8 - Loss: 0.210145
```

```
Epoch 1 Summary:
```

```
  Average Loss: 0.364230  
  Time: 3254.45s (0 img/s)  
  Memory: 892.78 MB (Peak: 892.78 MB)
```

```
[INFO] Saving 1 reconstruction samples to /content/PP-Final_Project/models/saved_weights  
      ✓ Saved 1 reconstruction samples to /content/PP-Final_Project/models/saved_weights/epoch_  
1_sample_*.ppm
```

```
--- Epoch 2/2 ---
```

```
Batch 1/8 - Loss: 0.181262  
Batch 2/8 - Loss: 0.177880  
Batch 3/8 - Loss: 0.157782  
Batch 4/8 - Loss: 0.137341  
Batch 5/8 - Loss: 0.134940  
Batch 6/8 - Loss: 0.146345  
Batch 7/8 - Loss: 0.146580  
Batch 8/8 - Loss: 0.132770
```

```
Epoch 2 Summary:
```

```
  Average Loss: 0.151863  
  Time: 3253.67s (0 img/s)  
  Memory: 892.78 MB (Peak: 892.78 MB)
```

```
[INFO] Saving 1 reconstruction samples to /content/PP-Final_Project/models/saved_weights  
      ✓ Saved 1 reconstruction samples to /content/PP-Final_Project/models/saved_weights/epoch_  
2_sample_*.ppm
```

```
=== Results ===
```

```
Time: 6869.4s | Loss: 0.364230 -> 0.151863 (-58.3%)  
Final weights saved to: /content/PP-Final_Project/models/saved_weights/cpu_final.bin  
Metrics saved to: /content/PP-Final_Project/models/saved_weights/epoch_metrics_cpu.txt
```

```
In [7]: !./build/extract_features_cpu ./models/saved_weights/cpu_final.bin
```

```
Feature Extraction (CPU)
Loading weights from ./models/saved_weights/cpu_final.bin
```

```
=== Extracting Training Features ===
Loading CIFAR-10 training data...
  Loaded data_batch_1.bin
  Loaded data_batch_2.bin
  Loaded data_batch_3.bin
  Loaded data_batch_4.bin
  Loaded data_batch_5.bin
Training data loaded: 50000 images
Processing 500 training images...
  Processed 128 / 500
  Processed 256 / 500
  Processed 384 / 500
  Processed 500 / 500
Training features: 500 x 8192
Time: 669.414s
Saved: /content/PP-Final_Project/models/saved_weights/train_features_cpu.bin
```

```
=== Extracting Test Features ===
Loading CIFAR-10 test data...
Test data loaded: 10000 images
Processing 500 test images...
  Processed 128 / 500
  Processed 256 / 500
  Processed 384 / 500
  Processed 500 / 500
Test features: 500 x 8192
Time: 666.279s
Saved: /content/PP-Final_Project/models/saved_weights/test_features_cpu.bin
```

```
=== Feature Extraction Complete ===
```

```
In [8]: !./build/train_svm_cpu
```

SVM Training using LIBSVM (CPU)

=====

```
Loading training features...
  500 samples x 8192 features
Loading CIFAR-10 training data...
  Loaded data_batch_1.bin
  Loaded data_batch_2.bin
  Loaded data_batch_3.bin
  Loaded data_batch_4.bin
  Loaded data_batch_5.bin
Training data loaded: 50000 images
Loading test features...
  500 samples x 8192 features
Loading CIFAR-10 test data...
Test data loaded: 10000 images
```

SVM Parameters:

```
Kernel: RBF
C: 10
Gamma: 0.00012207 (auto)
```

```
Preparing training data...
Training SVM...
.*
optimization finished, #iter = 115
nu = 0.490784
obj = -381.180371, rho = 1.226559
nSV = 64, nBSV = 40
*
optimization finished, #iter = 88
nu = 0.930414
obj = -769.279234, rho = -1.451723
nSV = 102, nBSV = 88
.*
optimization finished, #iter = 159
nu = 0.561320
obj = -468.842292, rho = 2.397648
nSV = 82, nBSV = 48
.*
optimization finished, #iter = 123
nu = 0.770408
obj = -565.096215, rho = 1.146929
nSV = 85, nBSV = 63
.*
optimization finished, #iter = 190
nu = 0.735668
obj = -531.063150, rho = 2.306014
nSV = 90, nBSV = 56
.*
optimization finished, #iter = 113
nu = 0.375543
obj = -238.488902, rho = 0.647248
nSV = 45, nBSV = 23
.*
optimization finished, #iter = 129
nu = 0.801722
obj = -621.103030, rho = 2.587473
nSV = 90, nBSV = 69
.*
optimization finished, #iter = 110
nu = 0.804767
```

```
obj = -583.254462, rho = 0.446355
nSV = 85, nBSV = 63
.*
optimization finished, #iter = 119
nu = 0.465572
obj = -325.094921, rho = 0.978829
nSV = 58, nBSV = 35
.*
optimization finished, #iter = 127
nu = 0.463754
obj = -337.204957, rho = -1.323332
nSV = 63, nBSV = 34
.*
optimization finished, #iter = 125
nu = 0.665716
obj = -606.582732, rho = 2.149426
nSV = 94, nBSV = 70
.*
optimization finished, #iter = 134
nu = 0.540291
obj = -356.584751, rho = -1.215969
nSV = 68, nBSV = 40
.*
optimization finished, #iter = 157
nu = 0.610531
obj = -453.847735, rho = 0.326477
nSV = 77, nBSV = 46
.*
optimization finished, #iter = 122
nu = 0.611334
obj = -458.495026, rho = -0.034816
nSV = 75, nBSV = 47
.*
optimization finished, #iter = 139
nu = 0.581249
obj = -439.769517, rho = 1.884258
nSV = 74, nBSV = 45
.*
optimization finished, #iter = 107
nu = 0.426260
obj = -280.774627, rho = -0.182289
nSV = 55, nBSV = 28
.*
optimization finished, #iter = 127
nu = 0.616398
obj = -484.335206, rho = -0.310755
nSV = 82, nBSV = 53
.*
optimization finished, #iter = 169
nu = 0.536431
obj = -423.068706, rho = 2.573865
nSV = 78, nBSV = 47
.*
optimization finished, #iter = 104
nu = 0.796624
obj = -623.864934, rho = 2.732467
nSV = 84, nBSV = 65
.*
optimization finished, #iter = 128
nu = 0.758295
obj = -577.725579, rho = 2.979905
nSV = 86, nBSV = 62
```

```

.*
optimization finished, #iter = 96
nu = 0.416715
obj = -279.581547, rho = 1.272549
nSV = 49, nBSV = 33
.*
optimization finished, #iter = 135
nu = 0.782226
obj = -604.356068, rho = 3.340265
nSV = 86, nBSV = 65
.*
optimization finished, #iter = 94
nu = 0.806344
obj = -551.929268, rho = 1.356433
nSV = 80, nBSV = 63
*
optimization finished, #iter = 91
nu = 0.500311
obj = -388.282419, rho = 0.672598
nSV = 59, nBSV = 44
.*
optimization finished, #iter = 163
nu = 0.549635
obj = -407.631841, rho = -2.088623
nSV = 77, nBSV = 43
.*
optimization finished, #iter = 170
nu = 0.630286
obj = -501.456797, rho = -0.453954
nSV = 87, nBSV = 55
.*
optimization finished, #iter = 163
nu = 0.605461
obj = -466.066404, rho = -0.814669
nSV = 81, nBSV = 50
.*
optimization finished, #iter = 154
nu = 0.620626
obj = -493.099265, rho = -0.299255
nSV = 84, nBSV = 52
.*
optimization finished, #iter = 140
nu = 0.540255
obj = -385.970672, rho = -1.672590
nSV = 71, nBSV = 45
.*
optimization finished, #iter = 177
nu = 0.620870
obj = -523.151560, rho = -1.208878
nSV = 89, nBSV = 54
.*
optimization finished, #iter = 137
nu = 0.811256
obj = -528.039104, rho = 2.224598
nSV = 87, nBSV = 64
*
optimization finished, #iter = 85
nu = 0.493700
obj = -305.730828, rho = 1.109288
nSV = 54, nBSV = 33
*
optimization finished, #iter = 80

```

```

nu = 0.854528
obj = -590.332995, rho = 3.398399
nSV = 83, nBSV = 70
.*
optimization finished, #iter = 101
nu = 0.745779
obj = -476.267059, rho = 0.026491
nSV = 72, nBSV = 53
.*
optimization finished, #iter = 126
nu = 0.607901
obj = -423.940582, rho = 1.439268
nSV = 69, nBSV = 42
.*
optimization finished, #iter = 107
nu = 0.558983
obj = -357.040902, rho = -1.520215
nSV = 64, nBSV = 41
.*
optimization finished, #iter = 138
nu = 0.794261
obj = -545.374838, rho = 0.334966
nSV = 88, nBSV = 62
.*
optimization finished, #iter = 107
nu = 0.704386
obj = -476.052025, rho = -1.703901
nSV = 76, nBSV = 52
.*
optimization finished, #iter = 146
nu = 0.612167
obj = -461.782104, rho = -1.247790
nSV = 75, nBSV = 47
.*
optimization finished, #iter = 108
nu = 0.542716
obj = -360.414309, rho = 2.389845
nSV = 60, nBSV = 38
.*
optimization finished, #iter = 96
nu = 0.402853
obj = -231.116571, rho = -0.602830
nSV = 47, nBSV = 25
.*
optimization finished, #iter = 143
nu = 0.793664
obj = -616.987169, rho = -0.170642
nSV = 90, nBSV = 67
.*
optimization finished, #iter = 92
nu = 0.855880
obj = -597.056891, rho = -1.457500
nSV = 81, nBSV = 65
.*
optimization finished, #iter = 152
nu = 0.633333
obj = -460.423963, rho = -1.340935
nSV = 74, nBSV = 49
*
optimization finished, #iter = 88
nu = 0.482867
obj = -314.177317, rho = -0.349510

```


nSV = 57, nBSV = 34

Total nSV = 493

Training completed in 14.0228s

Evaluating on training set...

Training accuracy: 62.8%

Evaluating on test set...

Test accuracy: 30.2%

Saving predictions...

Predictions saved: /content/PP-Final_Project/models/saved_weights/test_predictions.txt

Confusion Matrix:

True\Pred	airplane	automobile	bird	cat	deer	dog	frog	horse	ship	truck
airplane	22	4	5	0	1	1	3	3	11	7
automobile	0	18	0	1	1	0	6	1	6	8
bird	8	1	9	3	15	2	5	2	5	1
cat	1	3	6	4	9	4	15	2	0	5
deer	5	1	8	0	18	0	4	1	1	2
dog	4	4	14	5	7	4	6	3	0	1
frog	1	2	5	1	21	0	21	1	1	1
horse	3	6	4	2	8	2	5	8	2	7
ship	19	4	1	0	1	1	1	0	22	8
truck	5	9	1	0	2	0	3	3	8	25

Per-Class Accuracy:

airplane : 38.60%
automobile : 43.90%
bird : 17.65%
cat : 8.16%
deer : 45.00%
dog : 8.33%
frog : 38.89%
horse : 17.02%
ship : 38.60%
truck : 44.64%

Saving results...

Results saved to: /content/PP-Final_Project/models/saved_weights/svm_results.txt

=== SVM Training Complete ===

Total runtime: 35.3749s

3.3 Display Reconstruction Samples

Visualize the original vs reconstructed images saved during training to assess model quality.

```
In [ ]: ### 3.3 Display Reconstruction Samples

import os
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
from glob import glob

# Find all reconstruction sample images
sample_dir = 'models/saved_weights'
sample_files = sorted(glob(f'{sample_dir}/epoch*_sample_.ppm'))

if sample_files:
    print("=" * 70)
```

```

print("RECONSTRUCTION SAMPLES")
print("=" * 70)
print(f"\nFound {len(sample_files)} reconstruction sample(s)")

# Display all samples
num_samples = len(sample_files)
fig, axes = plt.subplots(num_samples, 1, figsize=(14, 5 * num_samples))

# Handle single sample case
if num_samples == 1:
    axes = [axes]

for idx, filepath in enumerate(sample_files):
    img = mpimg.imread(filepath)
    axes[idx].imshow(img)
    axes[idx].axis('off')

    # Extract epoch number from filename
    filename = os.path.basename(filepath)
    epoch_num = filename.split('_')[1]
    axes[idx].set_title(f'Epoch {epoch_num}: Original (Left) vs Reconstructed (Right)',
                        fontsize=14, fontweight='bold', pad=10)

plt.tight_layout()
plt.savefig(f'{sample_dir}/reconstruction_comparison.png', dpi=150, bbox_inches='tight')
plt.show()

print(f"\n✓ Displayed {len(sample_files)} reconstruction sample(s)")
print(f"✓ Combined visualization saved to {sample_dir}/reconstruction_comparison.png")
else:
    print("⚠ No reconstruction samples found")
    print(f"Expected files matching: {sample_dir}/epoch_*_sample_*.ppm")

```

In []: *### 3.6.3 Visualize Classification Results*

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

pred_file = 'models/saved_weights/test_predictions.txt'

if os.path.exists(pred_file):
    # Load predictions (skip header)
    data = np.loadtxt(pred_file, skiprows=1, dtype=int)
    y_true = data[:, 0]
    y_pred = data[:, 1]

    # Calculate accuracy
    accuracy = (y_true == y_pred).sum() / len(y_true)

    print("=" * 70)
    print(f"OVERALL TEST ACCURACY: {accuracy*100:.2f}%")
    print("=" * 70)
    print()

    # CIFAR-10 class names
    class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
                    'dog', 'frog', 'horse', 'ship', 'truck']

    # Confusion matrix
    cm = confusion_matrix(y_true, y_pred)

```

```

# Plot confusion matrix
fig, ax = plt.subplots(figsize=(12, 10))
disp = ConfusionMatrixDisplay(cm, display_labels=class_names)
disp.plot(ax=ax, cmap='Blues', values_format='d')
plt.title(f'Confusion Matrix - Test Accuracy: {accuracy*100:.2f}%',
          fontsize=14, fontweight='bold')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.savefig('models/saved_weights/confusion_matrix_cpu.png', dpi=150, bbox_inches='tight')
plt.show()

print("✓ Confusion matrix saved")

# Per-class accuracy
per_class_acc = []
for i in range(len(class_names)):
    class_mask = (y_true == i)
    if class_mask.sum() > 0:
        acc = (y_pred[class_mask] == i).sum() / class_mask.sum()
        per_class_acc.append(acc * 100)
    else:
        per_class_acc.append(0)

# Bar plot
fig, ax = plt.subplots(figsize=(12, 6))
bars = ax.bar(class_names, per_class_acc, color='steelblue', alpha=0.7)
ax.set_ylabel('Accuracy (%)', fontsize=12)
ax.set_xlabel('Class', fontsize=12)
ax.set_title('Per-Class Accuracy', fontsize=14, fontweight='bold')
ax.axhline(y=accuracy*100, color='red', linestyle='--', linewidth=2,
           label=f'Overall: {accuracy*100:.2f}%')
ax.legend()
ax.grid(True, alpha=0.3, axis='y')
plt.xticks(rotation=45, ha='right')

# Add value labels on bars
for bar, acc in zip(bars, per_class_acc):
    height = bar.get_height()
    ax.text(bar.get_x() + bar.get_width()/2., height,
            f'{acc:.1f}%', ha='center', va='bottom', fontsize=9)

plt.tight_layout()
plt.savefig('models/saved_weights/per_class_accuracy_cpu.png', dpi=150, bbox_inches='tight')
plt.show()

print("\n✓ Per-class accuracy visualization saved")

else:
    print(f"⚠ Predictions file not found: {pred_file}")

```

```

In [ ]: # Parse and display SVM results
import os

svm_results_file = 'models/saved_weights/svm_results.txt'

if os.path.exists(svm_results_file):
    print("=" * 70)
    print("SVM CLASSIFICATION RESULTS")
    print("=" * 70)
    print()

    with open(svm_results_file, 'r') as f:

```

```
        content = f.read()
        print(content)

    print("=" * 70)
else:
    print(f"⚠️ SVM results file not found: {svm_results_file}")
```

```
In [ ]: # Build CPU test executables (optional)
!cd build && make test_dataloader test_layers_cpu test_autoencoder_cpu -j$(nproc)
print("\n✓ CPU test executables compiled")
```

```
In [ ]: # Run CPU tests (optional)
print("=" * 70)
print("RUNNING CPU TESTS")
print("=" * 70)
print()

print("Test 1: Dataloader Test")
print("-" * 70)
!./build/test_dataloader

print("\n\nTest 2: CPU Layers Test")
print("-" * 70)
!./build/test_layers_cpu

print("\n\nTest 3: CPU Autoencoder Test")
print("-" * 70)
!./build/test_autoencoder_cpu

print()
print("=" * 70)
print("✓ All CPU tests completed")
print("=" * 70)
```